Simulating a Particle Detector and Particle Decay in C++

Aidan Hughes

11306492

School of Physics and Astronomy

University of Manchester

Third Year Code Report

May 2026

Abstract

The aim of this project was to utilize object oriented programming practices to create a particle detector and particle decay simulators in C++. It consists of two main inheritance chains, one for the particle classes and another for sub-detector classes. These classes, and their respective container classes ParticleBunch and Detector, practice encapsulation by storing data members as private or protected with access via public member functions. The use of virtual and pure virtual functions in the abstract parent classes of the hierarchy enables polymorphism in the child classes.

1 Introduction

The aim of this project was to create a C++ library that simulates a particle detector and its detecting functionality. As an addition, this code also aims to simulate particle decays within an accelerator. Particle detectors, such as ATLAS or the CMS, compose of multiple sub-detector types, each of which measure different types of particles.
The tracker, the first sub-detector a particle will pass through, uses magnetic fields to measure particle paths, so can only measure charged particles. Next are the calorimeters, which are split into two, an electromagnetic and a hadronic calorimeter. The electromagnetic calorimeter causes electrons and photons to trigger a particle shower which is measured to finds their energy. The hadronic calorimeter does the same but for protons and neutrons. Finally, due to muons high energy, they are not turned into showers by the calorimeters, and pass through to the Muon Spectrometer sub-detector. Using information about which sub-detectors were triggered, the type of particle that passed through can be deduced. These sub-detectors also have a detection efficiency associated with them, which compares the number of particles detected to the number produced during an event, usually represented as a percentage.
Along with the particles mentioned above, tau and Higgs particles are also found in accelerators, but decay quickly to other particles. This project focuses on the decays to muon and neutrinos for tau and di-photon decay for the Higgs. All particles have a four-momentum, which is a relativistically conserved property, akin to conservation of three-momentum in classical mechanics. MAny particles also have masses, charges or even type specific conservation numbers like lepton number.
This was done through two inheritance chains, one for the sub-detectors with the base class SubDetector and one for particles with base class Particle.

2 Code Implementation

Object oriented programming (OOP) is built on four pillars: encapsulation, inheritance, polymorphism and abstraction.
PLAN OOP EXAMPLES encapsulation - all classes, particle probaly best inheritance - particle hierarchy polymorphism static - overloaded function - constuctors dynamic - info() function on particles - detector flags abstraction - particle type enum class
FEATURES USED enum class for particle type - validation in one place - easier to scale switch cases than if enables - re used for Neutrino flavour deque to do particle decay unordered map to do key value pairs rand() for efficiency / misdetection calulation

3 Results

```
1          int main()
2          {
```

```
3                   return 0;
4           }
```

4 Discussion

errors on detected value. Extra abstraction in detect function.